

Uwagi - (przeczytaj dokładnie!)

- Można korzystać z własnych (odręcznych) notatek oraz manuala.
- W przesyłanych komunikatach nie używamy polskich znaków.
- Pliki z programami należy przesłać na Moodle. Nazwy plików podane są przy poszczególnych zadaniach (w miejsce `NazwiskoImie` należy wpisać swoje nazwisko i imię). Proszę nazywać pliki dokładnie tak jak jest to opisane przy zadaniu.
- Poniższe zadania są do wyboru, tj., należy wybrać jedno z nich i tylko je rozwiązywać. Zadania przedstawione są w kolejności rosnącego stopnia trudności i rosnącej maksymalnej liczby punktów. Proszę zwrócić uwagę, że:
 - każde z zadań o numerach 1 i 2 wymaga napisania 2 programów (para klient-serwer),
 - każde z zadań o numerach 3, 4, 5 wymaga napisania 4 programów (dwie pary klient-serwer)!
- Wszystkie zadania polegają na napisaniu pary (par) programów klient i serwer komunikujących się przy pomocy rodziny protokołów INET (IPv4); typ protokołu będzie podany w poszczególnych zadaniach.
- Zadania powinny być robione ściśle zgodnie ze specyfikacją (włączając w to wygląd „interfejsu”, w tym estetyczne wyrównania!) – za odstępstwa będą obcinane punkty.
- Pamiętajmy o obsłudze najważniejszych błędów (np. błędne dane od użytkownika) oraz o „posprzątaniu”.

Zadania

1. Maks. 11,3 pkt. (~ ocena 3); nazwy plików: `NazwiskoImie-serwer3.c`, `NazwiskoImie-klient3.c`

Typ protokołu: bezpołączeniowy (UDP).

Serwer

- uruchamiany bez argumentów, zaraz po uruchomieniu wyświetla napis: `Czekam...`
- oczekuje na zgłoszenia klientów na porcie 8888,
- użytkownik może wyłączyć serwer poprzez naciśnięcie Ctrl-C; wówczas wyświetla się napis: `Konczymy`,
- klienci przysyłają wiadomości w postaci ciągów znaków;
- serwer zlicza przysłane wiadomości i każdą wiadomość, wraz z jej numerem, IP klienta i jego numerem portu wyświetla w postaci jak w przykładzie poniżej.

Klient

- uruchamiany z jednym argumentem: IP serwera
- wyświetla komunikat: `Podaj wiadomosc do serwera:` oraz w nowej linii znak zachęty `>`
- podaną przez użytkownika wiadomość tekstową przesyła do serwera na port 8888 i kończy pracę.

Przykładowe wykonanie serwera:

```
Czekam...
Wiadomosc nr 1 (IP: 158.75.112.120, port: 50997): Czesc!
Wiadomosc nr 2 (IP: 158.75.112.120, port: 52178): Ala ma kota
Wiadomosc nr 3 (IP: 158.75.2.10, port: 51004): 123
^C
Konczymy
```

Przykładowe wykonanie (jednego) klienta:

```
Podaj wiadomosc do serwera:
> Ala ma kota
```

2. Maks. 13,3 pkt. (~ ocena 3+); nazwy plików: `NazwiskoImie-serwer3+.c`, `NazwiskoImie-klient3.c+`

Typ protokołu: bezpołączeniowy (UDP).

Serwer

- uruchamiany bez argumentów, zaraz po uruchomieniu wyświetla napis: `Czekam...`
- oczekuje na zgłoszenia klientów na porcie 8888,
- użytkownik może wyłączyć serwer poprzez naciśnięcie Ctrl-C; wówczas wyświetla się napis: `Konczymy`,
- klienci przysyłają wiadomości w postaci ciągów znaków;
- serwer zlicza przysłane wiadomości i każdą wiadomość, wraz z jej numerem, IP klienta i jego numerem portu wyświetla w postaci jak w przykładzie poniżej.
- odsyła klientowi zduplikowany napis, który dostał od klienta (tj., np. klient przysłał `Ala ma kota`, to serwer ma odesłać `Ala ma kotaAla ma kota`) oraz wyświetla o tym informację zgodnie z przykładem poniżej.

Klient

- uruchamiany z jednym argumentem: IP serwera
- wyświetla komunikat: **Podaj wiadomosc do serwera:** oraz w nowej linii znak zachęty >
- podaną przez użytkownika wiadomość tekstową przesyła do serwera na port 8888
- czeka na odpowiedź serwera i wyświetla ją niżej po napisie **Odpowiedz serwera:**, a następnie kończy pracę.

Przykładowe wykonanie serwera:

```
Czekam...
Wiadomosc nr 1 (IP: 158.75.112.120, port: 50997): Czeszc!
Odsylam: Czeszc!Czeszc!
Wiadomosc nr 2 (IP: 158.75.112.120, port: 52178): Ala ma kota
Odsylam: Ala ma kotaAla ma kota
Wiadomosc nr 3 (IP: 158.75.2.10, port: 51004): 123
Odsylam: 123123
^C
Konczymy
```

Przykładowe wykonanie (jednego) klienta:

```
Podaj wiadomosc do serwera:
> Ala ma kota
Odpowiedz serwera: Ala ma kotaAla ma kota
```

3. Maks. 15,3 pkt. (~ ocena 4); nazwy plików: **NazwiskoImie-serwer4.c**, **NazwiskoImie-klient4.c** oraz **dodatkowo** należy zrobić Zadanie 2, a więc przesłać też pliki **NazwiskoImie-serwer3+.c**, **NazwiskoImie-klient3+.c**

Typ protokołu: połączeniowy (TCP).

Napisz parę klient-serwer realizującą to samo co programy w Zadaniu 2, ale przy wykorzystaniu protokołu TCP. Innymi słowy, należy przesłać 4 pliki: **NazwiskoImie-serwer4.c**, **NazwiskoImie-klient4.c** (realizujące wersję TCP) oraz **NazwiskoImie-serwer3+.c**, **NazwiskoImie-klient3+.c** (realizujące wersję UDP).

4. Maks. 17,3 pkt. (~ ocena 4+); nazwy plików: **NazwiskoImie-serwer4+.c**, **NazwiskoImie-klient4.c+** oraz **dodatkowo** należy zrobić Zadanie 2, a więc przesłać też pliki **NazwiskoImie-serwer3+.c**, **NazwiskoImie-klient3+.c**

Typ protokołu: połączeniowy (TCP).

Napisz parę klient-serwer realizującą to samo co programy w Zadaniu 2, ale

- przy wykorzystaniu protokołu TCP,
- klient uruchamiany z jednym argumentem: IP serwera lub nazwą domenową (wskazówka: `getaddrinfo()`),
- serwer powinien być napisany w wersji współbieżnej (pamiętajmy, że nie lubimy zombich!),
- po stronie serwera dodatkowo wyświetlamy PID procesu obsługującego danego klienta jak w przykładzie poniżej.

Przykładowe wykonanie serwera:

```
Czekam...
[PID 31236]: Wiadomosc nr 1 (IP: 158.75.112.120, port: 50997): Czeszc!
[PID 31236]: Odsylam: Czeszc!Czeszc!
[PID 31239]: Wiadomosc nr 2 (IP: 158.75.112.120, port: 52178): Ala ma kota
[PID 31239]: Odsylam: Ala ma kotaAla ma kota
[PID 31247]: Wiadomosc nr 3 (IP: 158.75.2.10, port: 51004): 123
[PID 31247]: Odsylam: 123123
^C
Konczymy
```

Przykładowe wykonanie (jednego) klienta:

```
Podaj wiadomosc do serwera:
> Ala ma kota
Odpowiedz serwera: Ala ma kotaAla ma kota
```

5. Maks. 20 pkt. (~ ocena 5); nazwy plików: **NazwiskoImie-serwer5.c**, **NazwiskoImie-klient5.c** oraz **dodatkowo** należy zrobić Zadanie 2, a więc przesłać też pliki **NazwiskoImie-serwer3+.c**, **NazwiskoImie-klient3+.c**

Typ protokołu: połączeniowy (TCP).

Napisz parę klient-serwer realizującą to samo co programy w Zadaniu 2, ale

- przy wykorzystaniu protokołu TCP,
- uruchamiany z jednym argumentem: IP serwera lub nazwą domenową (wskazówka: `getaddrinfo()`),
- serwer powinien być napisany w wersji współbieżnej (pamiętajmy, że nie lubimy zombich!),
- po stronie serwera dodatkowo wyświetlamy PID procesu obsługującego danego klienta jak w przykładzie poniżej.
- klient działa w pętli, tj. pobiera od użytkownika, wysyła do serwera i odbiera kolejne komunikaty dopóki użytkownik nie napisze <koniec> (serwer przetwarza kolejne komunikaty klienta w ten sam sposób jak w Zadaniu 2 – patrz przykład poniżej),
- serwer wykorzystuje semafor zliczający do zliczania aktywnych procesów potomnych.

Przykładowe wykonanie serwera:

```
Czekam...
[PID 31236]: Wiadomosc nr 1 (IP: 158.75.112.120, port: 50997): Czeszc!
[PID 31236]: Odsylam: Czeszc!Czeszc!
[PID 31239]: Wiadomosc nr 2 (IP: 158.75.112.120, port: 52178): Ala ma kota
[PID 31239]: Odsylam: Ala ma kotaAla ma kota
[PID 31247]: Wiadomosc nr 3 (IP: 158.75.2.10, port: 51004): 123
[PID 31247]: Odsylam: 123123
[PID 31239]: Wiadomosc nr 4 (IP: 158.75.112.120, port: 52178): A kot ma Ale
[PID 31239]: Odsylam: A kot ma AleaA kot ma Ale
[PID 31247]: Wiadomosc nr 5 (IP: 158.75.2.10, port: 51004): ?
[PID 31247]: Odsylam: ??
^C
```

Przykładowe wykonanie (jednego) klienta:

```
Podaj wiadomosc do serwera:
> Ala ma kota
Odpowiedz serwera: Ala ma kotaAla ma kota

> A kot ma Ale
Odpowiedz serwera: A kot ma AleA kot ma Ale

> <koniec>
```